

# MLE-STAR Reconstruction Report

A didactic guide to the OpenAI-compatible port

Branch: feature/reconstruct-mle-star-openai

Base branch: feature/issue-1.5-openai-port

Generated: 2026-06-17



# 1. What is this project and why does it matter?

This project is an autonomous machine-learning engineering system called MLE-STAR. In plain language: you give it a dataset and a task description, and it tries to design, train, debug, and improve a machine-learning solution for you without human intervention.

Think of a data-science team where each member has a specialty:

- One person reads the competition description and summarizes it.
- Another person researches which models work well for that type of data.
- A third person writes the first training script.
- A fourth person tries to combine two models into something stronger.
- A fifth person checks that the solution uses all available information.
- A sixth person runs ablation studies to find the most important parts of the code.
- A seventh person builds an ensemble of the best solutions.
- An eighth person prepares the final submission file.

MLE-STAR automates that entire team. Each specialty is implemented as an agent. An agent is just a function that sends a prompt to a large language model (LLM), receives a response, and stores the result in a shared notebook called AgentState.

## 1.1 Why are we rebuilding it from scratch?

The original MLE-STAR was built on two Google technologies:

- ADK (Agent Development Kit): a framework that orchestrates agents.
- Gemini: the LLM family that powers the agents.

In our current environment we cannot use Google ADK or Gemini. We must therefore rebuild MLE-STAR so that it runs entirely on the university's OpenAI-compatible API. An

OpenAI-compatible API is an endpoint that accepts the same HTTP requests as the official OpenAI API. That means we can use the standard `openai` Python library and simply change the base URL and API key.

## 1.2 What was already done before this reconstruction?

A teammate had already created a minimal OpenAI-compatible port in branch `feature/issue-1.5-openai-port`. That minimal port could connect to the university API and run a very short pipeline:

1. Propose one machine-learning model.
2. Generate one training script for that model.
3. Run the script and parse the validation score.
4. Create a `submission.csv` file.

That is useful, but it is not MLE-STAR. The full MLE-STAR system is much deeper: it proposes many models, merges the best ones, studies which code blocks matter, refines them, builds an ensemble, and only then submits. This reconstruction adds all of those missing pieces.

# 2. Concepts you must understand before reading the code

## 2.1 Large Language Model (LLM)

A large language model is a program trained to predict text. You send it a prompt (a block of text) and it returns a continuation. In this project the LLM is asked to write Python code, summarize text, and make decisions. We use the university model `meta-llama-3.1-8b-instruct`, which is smaller than models like GPT-4 or Gemini-Pro. That means it is cheaper and faster, but it also makes more mistakes and ignores instructions more often. The whole reconstruction is designed with that limitation in mind.

## 2.2 Agent

An agent is a Python function that prepares a prompt, calls the LLM, and processes the response. For example, the model retriever agent prepares a prompt that says 'Propose two machine-learning models for this task', sends it to the LLM, and parses the returned JSON into Python dictionaries.

## 2.3 AgentState

AgentState is a Python dictionary that stores everything the pipeline needs. Think of it as a shared notebook that every agent can read and write. Examples of keys:

- task\_description: the text read from task\_description.txt.
- task\_summary: a shorter summary produced by the LLM.
- init\_1\_model\_1: the first model candidate for task\_id 1.
- init\_code\_1\_1: the Python code generated for that candidate.
- init\_code\_exec\_result\_1\_1: the result of running that code (returncode, stdout, stderr, score).
- train\_code\_0\_1: the selected solution after initialization.
- best\_refined\_code: the best solution found during refinement.
- total\_tokens\_spent: cumulative LLM token count.

The naming convention is important. The helper code\_util uses the agent name to figure out which keys to read and write.

## 2.4 Prompt

A prompt is a string that tells the LLM what to do. Prompts live in prompt.py files. They contain placeholders such as {task\_description} and {code} that are filled at runtime using Python's .format() method. Writing good prompts is the most important part of making the system work.

## 2.5 Runner

The runner is the replacement for Google ADK. It lives in machine\_learning\_engineering/runner.py and provides three simple functions:

- run\_agent(state, agent\_name, instruction, ...): sends a prompt to the LLM and stores the response.
  - run\_sequential(state, agents): runs several agents one after another.
  - run\_loop(state, agent, max\_iterations): repeats one agent up to max\_iterations times.
- run\_agent builds an OpenAI chat-completion request, sends it, extracts the text response, and adds the token count to state['total\_tokens\_spent'].

## 2.6 Code execution and debugging

code\_util.run\_python\_code writes a string of Python code to a file and executes it with subprocess. It captures stdout, stderr, return code, and execution time. debug\_util.run\_and\_debug is the high-level helper that asks the LLM for code, runs it, and, if it fails, asks the LLM to fix the error. This debug loop is what makes the system autonomous.

## 2.7 Validation score

Every generated training script must print a line like:

Final Validation Performance: 245.52

The runner reads the script's stdout, finds that line, and extracts the number. That number is used to rank solutions. If the line is missing, the run is considered a failure.

# 3. What the repository looks like

Before changing anything, you need to understand the layout. Here is the repository after the reconstruction, with an explanation of each important file.

```

machine_learning_engineering/
__init__.py          # Loads .env and creates the OpenAI client
agent.py             # Top-level pipeline
runner.py            # OpenAI-compatible runner (replaces ADK)
prompt.py            # Legacy frontdoor prompts
shared_libraries/
  config.py          # DefaultConfig dataclass
  code_util.py       # Code execution and state-key helpers
  common_util.py     # Small helpers
  debug_util.py      # run_and_debug helper
  debug_prompt.py    # Prompt used by the debug agent
  data_leakage_prompt.py # Prompts for leakage detection
  check_leakage_util.py # OpenAI leakage checker (rewritten)
sub_agents/
  initialization/
    agent.py         # Initialization pipeline
    prompt.py        # Initialization prompts
  refinement/
    agent.py         # Refinement pipeline
    prompt.py        # Refinement prompts
  ensemble/
    agent.py         # Ensemble pipeline
    prompt.py        # Ensemble prompts
  submission/
    agent.py         # Submission pipeline
    prompt.py        # Submission prompts
tasks/
  california-housing-prices/
    task_description.txt
    train.csv
    test.csv
workspace/           # Generated files

```

### 3.1 The top-level pipeline file

machine\_learning\_engineering/agent.py is the entry point. When you run `python3 -m machine_learning_engineering.agent`, it creates an empty `AgentState`, fills basic configuration, and then calls the four sub-pipelines in order. Finally it saves the full state to `final_state.json`.

### 3.2 The runner

machine\_learning\_engineering/runner.py is the heart of the port. It no longer depends on Google ADK. It imports the OpenAI client created in `__init__.py` and uses it to call the university endpoint. This file was not modified during the reconstruction because the minimal port already had a working runner.

### 3.3 Shared libraries

The `shared_libraries` folder contains utilities used by many agents. `code_util.py` is especially important because it maps agent names to state keys. For example, for agent name `model_eval_1_2`, `code_util` knows that the code key is `init_code_1_2` and the result key is `init_code_exec_result_1_2`.

### 3.4 Sub-agents

Each `sub_agents/<name>/` folder contains one phase of the pipeline. `agent.py` implements the logic and `prompt.py` contains the prompts. The reconstruction added two new folders (`refinement` and `ensemble`) and extended the existing ones (`initialization` and `submission`).

## 4. Phase 0 - Preparation

### 4.1 Goal

Phase 0 prepares the branch and the configuration for the first end-to-end run. The key idea is to start conservative: use small loops and few candidates so that the pipeline finishes quickly and cheaply. Once the pipeline is stable, you can increase the values.

### 4.2 What was changed

Only one file was edited in this phase: `machine_learning_engineering/shared_libraries/config.py`. It contains a dataclass called `DefaultConfig`. We changed the default values to match the reconstruction plan:

```

agent_model = os.environ.get(
    "ROOT_AGENT_MODEL", "meta-llama-3.1-8b-instruct"
)

num_solutions = 1
num_model_candidates = 2
max_retry = 2
max_debug_round = 2
max_rollback_round = 2
inner_loop_round = 1
outer_loop_round = 1
ensemble_loop_round = 1
use_data_leakage_checker = True
use_data_usage_checker = True

```

### 4.3 Why these values

- num\_solutions=1 means the pipeline generates one solution path. Parallelism is disabled for the first stable run.
- num\_model\_candidates=2 gives the merger something to combine without exploding cost.
- max\_retry=2 and max\_debug\_round=2 give the small model two chances to recover from errors.
- inner\_loop\_round=1 and outer\_loop\_round=1 mean refinement runs one step with one alternative plan.
- ensemble\_loop\_round=1 means the ensemble tries one initial plan and one refined plan.
- The checkers are enabled because the plan explicitly requires them.

## 5. Phase 1 - Strengthen initialization

Initialization is the first big pipeline. Its job is to propose models, evaluate them, merge the best ones, verify data usage, check for leakage, and produce a starting solution called `train_code_0_{task_id}`. The minimal port only did the first two steps (propose one model, evaluate it). This phase adds everything else.

### 5.1 Step 1 - Prepare task and workspace

`prepare_task` copies all values from `DefaultConfig` into `AgentState` and reads `task_description.txt`. `create_workspace` creates directories for the `task_id` and copies the input data. If the directory already exists, it is deleted first so every run starts clean.

This step is important because every generated script expects its input in `./input` relative to its execution directory.

### 5.2 Step 2 - Summarize the task

The `task_summarization_agent` sends `SUMMARIZATION_AGENT_INSTR` to the LLM. The prompt contains the full task description and asks for a short summary. The summary is stored in `state['task_summary']` and is used by the model retriever agent.

```

SUMMARIZATION_AGENT_INSTR = """# Task description
{task_description}

# Your task
- Summarize this task description.
- Your summary will be used for searching recent effective models for {task_type}.

# Requirement
- We will directly use your response, so be simple and concise.
"""

```

### 5.3 Step 3 - Retrieve model candidates

The `model_retriever_agent` sends `MODEL_RETRIEVAL_INSTR` to the LLM. The prompt asks for `num_model_candidates` models and a short code snippet for each. The response should be a JSON list, but the small model often fails to follow the exact format. We therefore implemented a robust parser with three layers:

- First, try to extract a JSON list from the response and parse it with `ast.literal_eval`.

- If that fails, look for markdown Python blocks (``python ... ``) and treat each block as a candidate.
- If still nothing works, fall back to hard-coded scikit-learn defaults: RandomForestRegressor and GradientBoostingRegressor.

The parsed candidates are stored as `init_{task_id}_model_{j}`. A human-readable description is also written to `model_candidates/model_{j}.txt`.

Here is the robust parser that handles the small model's imperfect output:

```
def _parse_model_candidates(response_text, num_model_candidates):
    start_idx, end_idx = response_text.find('{', response_text.rfind('}') + 1)
    if start_idx != -1 and end_idx > start_idx:
        try:
            models = ast.literal_eval(response_text[start_idx:end_idx])
            if isinstance(models, list):
                return models[:num_model_candidates]
        except Exception:
            pass
    code_blocks = re.findall(r"``python\n(.*?)\n``", response_text, re.DOTALL)
    models = []
    for block in code_blocks[:num_model_candidates]:
        name_match = re.search(r"from sklearn\.\\w+ import (\\w+)", block)
        name = name_match.group(1) if name_match else 'Model'
        models.append({'model_name': name, 'example_code': block})
    return models if models else _DEFAULT_CANDIDATES[:num_model_candidates]
```

## 5.4 Step 4 - Evaluate every candidate

`run_model_eval_for_all_candidates` loops over every parsed candidate and calls `debug_util.run_and_debug` with agent name `model_eval_{task_id}_{model_id}`. The `MODEL_EVAL_INSTR` prompt tells the LLM exactly what to do:

- Use the proposed model.
- Load `train.csv` from `./input`.
- Split it into train and validation sets.
- Train the model and evaluate it.
- Print 'Final Validation Performance: <score>'.
- Do not use `test.csv` for the validation score.

The generated code is stored under `init_code_{task_id}_{model_id}`. It is then executed and the result is stored under `init_code_exec_result_{task_id}_{model_id}`. If execution fails, the debug loop tries to fix the error using `BUG_REFINE_INSTR`.

## 5.5 Step 5 - Rank candidates

`rank_candidate_solutions` collects every successful `init_code_exec_result`, sorts by score, and stores the best as `base_solution_{task_id}`. The lower-is-better flag in config is respected: for regression metrics like RMSE, a lower score is better. This best candidate becomes the starting point for merging.

## 5.6 Step 6 - Merge candidates

`run_merger_and_debug_loop` takes the best candidate and tries to combine it with each remaining candidate. It uses agent name `merger_{task_id}_{reference_idx}` and the `CODE_INTEGRATION_INSTR` prompt. The prompt shows the base solution and a reference solution and asks the LLM to produce a single script that ensembles both models.

We made two important fixes here. First, we added 'Do NOT load test.csv' to the prompt because the small model sometimes tried to make test predictions during the merge, which broke feature names. Second, we skip any reference candidate that did not generate valid code.

## 5.7 Step 7 - Select best merged solution

`select_best_solution` promotes the best merged (or base) solution to `train_code_0_{task_id}` and writes it to `train0.py`. We added a safety net: if the selected merged solution failed to execute, the function falls back to the best individual candidate. It also stores `best_initial_code` and `best_initial_score` so that the submission phase can always fall back to a known-good solution.

The fallback looks like this:

```

response = state.get(f"merger_code_{{task_id}}_{{best_idx}}", "")
result_dict = state.get(f"merger_code_exec_result_{{task_id}}_{{best_idx}}", {})

if not result_dict or result_dict.get("returncode", 1) != 0:
    performance_results = state.get(f"performance_results_{{task_id}}", [])
    if performance_results:
        response = performance_results[0][1]
        result_dict = performance_results[0][2]

state["best_initial_code"] = code_text
state["best_initial_score"] = result_dict.get("score")

```

## 5.8 Step 8 - Data-use checker

run\_check\_data\_use\_and\_debug\_loop sends the selected solution to the LLM with CHECK\_DATA\_USE\_INSTR. The prompt asks: 'Does this code use all the information provided? If not, improve it.' If the LLM decides all information is used, it replies with the exact sentence 'All the provided information is used.' and the pipeline moves on. Otherwise the LLM returns improved code, which is executed and scored.

We added a clarification to prevent a common hallucination: 'Only use columns that actually exist in train.csv and test.csv. Do NOT invent columns such as ocean\_proximity.' Without this, the small model often invented features that do not exist.

## 5.9 Step 9 - Data-leakage checker

Data leakage means using information from the test set or validation set during training. For example, fitting a scaler on the full dataset (including test samples) and then using it on training data is leakage.

Every successful code execution now optionally runs check\_and\_fix\_leakage. This function sends the code to the LLM with CHECK\_LEAKAGE\_INSTR. The LLM must answer either 'No Data Leakage' or 'Yes Data Leakage' plus the problematic code block. If leakage is detected, a refine agent rewrites the block and the code is re-executed.

The original check\_leakage\_util.py depended on Google ADK types and agents. It was completely broken in the OpenAI port. We rewrote it from scratch to use our runner and AgentState.

In debug\_util.py, the leakage check is triggered after a successful execution:

```

result = _get_result(state, agent_name)
if _is_successful(result):
    check_leakage_util.check_and_fix_leakage(
        state,
        agent_name,
        max_iterations=state.get("max_retry", 1),
    )
    return

```

# 6. Phase 2 - Implement refinement

Refinement is where the system tries to improve the initial solution. The idea comes from ablation studies: remove one part of the code at a time, see how much the score drops, and then focus on improving the parts that matter most.

## 6.1 The refinement loop

run\_refinement\_pipeline is driven by outer\_loop\_round and inner\_loop\_round. With the conservative config (outer=1, inner=1), the pipeline does the following for task\_id 1:

1. Run ablation\_agent.
2. Run ablation\_summary\_agent.
3. Run init\_plan\_agent.
4. Run init\_plan\_implement\_agent.
5. Run plan\_refine\_agent (inner loop).
6. Run plan\_implement\_agent (inner loop).
7. Select the best improvement and promote it to the next step.

The current solution at each step is stored under train\_code\_{step}\_{task\_id}. The best solution ever seen is stored in best\_refined\_code/best\_refined\_score.



## 6.2 Ablation agent

The `ablation_agent` receives the current solution and the task description. We deliberately simplified the prompt because the small model cannot reliably generate complex multi-ablation scripts. The simplified ablation trains the full solution, then trains ONE simplified variant by removing the most complex preprocessing block, and reports both scores.

If the ablation script fails (for example due to timeout or syntax error), the pipeline continues anyway. The summary agent receives the error output and produces a generic recommendation, which is enough to keep the loop moving.

## 6.3 Plan agents

`init_plan_agent` reads the current solution and the ablation summary and proposes a JSON object with two fields: `plan` (a text description of what to improve) and `code_block` (the exact block to modify). `plan_refine_agent` does the same but must propose a different plan.

Because the small model often returns malformed JSON, we added `_extract_json_block` and a default plan fallback. If the LLM does not return valid JSON, the system uses a safe generic plan: 'Simplify the model training block and use a more robust validation setup.'

## 6.4 Plan implement agent

`plan_implement_agent` is the only agent in refinement that produces executable code. The prompt shows the current solution, the plan, and the code block, and asks the LLM to return the complete improved solution.

The original design in `debug_util.get_code_from_response` assumed the LLM returned only the replacement block. That works with large models like Gemini-Pro but fails with the small university model, which usually returns the full script. We therefore changed the logic to treat the LLM response as the complete improved solution.

We also added strict constraints to the prompt:

- Use only `train.csv` from `./input` for training and validation. Do NOT load `test.csv`.
- Do NOT change which columns are used as features.
- Do NOT use `GridSearchCV` or nested pipelines unless the original code already uses them correctly.

After each `plan_implement`, `_select_best_improvement` compares the new score with the best refined score. If the new code is worse or broken, the old best solution is kept.

The change in `debug_util.get_code_from_response` looks like this:

```
elif agent_name.startswith("plan_implement"):
    # The LLM returns the complete improved solution.
    new_code = code
    if "debug" not in agent_name:
        task_id = agent_name.split("_")[-1]
        step = state.get(f"refine_step_{{task_id}}", 0)
        inner_iter = state.get(f"inner_iter_{{task_id}}", 0)
        state[f"train_code_{{step}}_{{task_id}}"] = new_code
        state[f"train_code_improve_{{inner_iter}}_{{step}}_{{task_id}}"] = new_code
```

## 7. Phase 3 - Implement ensemble

The ensemble agent tries to combine multiple solutions into one stronger model. With `num_solutions=1` there is only one refined solution, so the ensemble usually combines the models inside that solution, for example by averaging two regressors.

### 7.1 Ensemble pipeline

`run_ensemble_pipeline` first creates an ensemble workspace under `workspace/<task>/ensemble` and copies the input data. Then it executes:

1. `init_ensemble_plan_agent` — proposes an ensemble strategy.
2. `init_ensemble_plan_implement_agent` — writes and executes `ensemble0.py`.
3. `ensemble_plan_refine_agent` — proposes a different strategy.
4. `ensemble_plan_implement_agent` — writes and executes `ensemble1.py`.
5. `_select_best_ensemble` — keeps the ensemble with the best validation score.

The ensemble prompts explicitly forbid assuming non-existent columns. Without this fix, the small

model repeatedly invented `ocean_proximity`.

## 7.2 Why ensemble can fail

The ensemble agent has to understand the feature columns used by the refined solution. The small model often assumes that the dataset has the same columns as a public California Housing example it saw during training, such as `ocean_proximity`. When that column is missing, the generated ensemble script crashes with a `KeyError`. The fallback mechanism in the submission agent ensures this crash does not break the final submission.

## 8. Phase 4 - Strengthen submission

The submission agent receives the best available solution and asks the LLM to adapt it so that it trains on the full training set and writes `./final/submission.csv`. We made two important changes in this phase.

- Best-solution selection: instead of always using `train_code_0_1`, the agent now picks the best among `best_refined_code`, `best_ensemble_code`, and `best_initial_code`. This guarantees that the strongest solution is submitted.
- Workspace preservation: the original submission agent deleted the ensemble workspace before recreating it. Now it only creates directories and copies input data if the workspace is empty, preserving files written by the ensemble agent.

The final submission prompt (`ADD_TEST_FINAL_INSTR`) tells the LLM to load `test.csv`, predict all samples, and save the result to `./final/submission.csv`.

The new selection logic in `submission/agent.py`:

```
def _select_final_solution(state) -> str:
    best_ensemble_code = state.get("best_ensemble_code", "")
    best_ensemble_score = state.get("best_ensemble_score")
    refined_score = state.get("best_refined_score")
    refined_code = state.get("best_refined_code", "")
    initial_code = state.get("best_initial_code", refined_code)

    best_code = refined_code
    best_score = refined_score
    if best_ensemble_code and best_ensemble_score is not None:
        lower = state.get("lower", True)
        if best_score is None or (
            (lower and best_ensemble_score <= best_score)
            or (not lower and best_ensemble_score >= best_score)
        ):
            best_code = best_ensemble_code
            best_score = best_ensemble_score

    if not best_code or best_score is None or best_score == 1e9:
        best_code = initial_code
    return best_code
```

## 9. Phase 5 - Integration and validation

The top-level `agent.py` now calls the four pipelines in order and saves the final state:

```
initialization_agent_module.run_initialization_pipeline(state)
refinement_agent_module.run_refinement_pipeline(state)
ensemble_agent_module.run_ensemble_pipeline(state)
submission_agent_module.run_submission_pipeline(state)
save_state(state)
```

The pipeline was validated end-to-end on the California Housing task. The final run produced:

Best score: 245.5250660916751

Submission return code: 0

Total tokens spent: 35029

The `submission.csv` file was generated successfully. In this run the best solution came from the initialization phase, demonstrating that the fallback mechanism works: refinement and ensemble run, but if they do not improve the score, the system keeps the best initial solution.

## 10. Errors encountered and how they were fixed

### 10.1 Could not read the PDF plan

The Read tool could not consume PDF input directly. The fix was to create a temporary virtual environment, install pdfplumber, extract the full text of the PDF to a plain text file, and then read that text file.

### 10.2 Disk full

The workspace filesystem reached 100% usage, causing `os.makedirs` to fail with `OSError: No space left on device`. The fix was to inspect `.venv` and discover that `torch` and `triton` had been installed. These packages are not needed for the scikit-learn-based California Housing task, so they were removed, freeing approximately 2.9 GB.

### 10.3 Model retriever produced unparsable output

The small model often ignored the strict JSON schema in `MODEL_RETRIEVAL_INSTR`, leaving `init_1_model_*` empty. The fix was a three-layer parser: try JSON list, then markdown Python blocks, then hard-coded scikit-learn defaults. This guarantees that the pipeline always has candidates to evaluate.

### 10.4 Score parsing failed

`extract_performance_from_text` originally split the performance line on `:` and tried to convert the last part directly to float. The small model sometimes added formatting or units. The fix was to use a regular expression to extract the last numeric token from the line.

### 10.5 Merger included test-set predictions

The merger sometimes generated code that loaded `test.csv` and made predictions, causing feature-name mismatches and invalid scores. The fix was to add `'Do NOT load test.csv'` to the merger prompt and to fall back to the best individual candidate in `select_best_solution` when the merged result is broken.

### 10.6 Plan implement broke working code

The original block-replacement design assumed the LLM returned only the replacement block. The small model returned the full script or introduced `GridSearchCV` errors. The fix was to accept the full script, add strict prompt constraints, and only update `best_refined_code` when the new code actually improves the score.

### 10.7 Ensemble hallucinated columns

The ensemble repeatedly referenced `ocean_proximity`, a column that does not exist in the provided files. The fix was to add explicit instructions to the ensemble prompts to use only columns present in `train.csv`.

### 10.8 Pipeline timeout

The first full run exceeded the 30-minute shell timeout. After tuning prompts and removing heavy packages, subsequent runs completed well within the limit.

## 11. How you would do this yourself

### 11.1 Environment setup

1. Make sure you are on branch `feature/reconstruct-mle-star-openai`.
2. Create a virtual environment and install the minimum dependencies:  

```
python3 -m venv .venv  
.venv/bin/pip install openai python-dotenv pydantic numpy pandas scikit-learn
```
3. Create a `.env` file with:  

```
OPENAI_API_BASE=https://chat-ai.academiccloud.de/v1  
OPENAI_API_KEY=your_token  
ROOT_AGENT_MODEL=meta-llama-3.1-8b-instruct
```

## 11.2 Phase 0

Open `config.py` and set the conservative values from section 4.2. Test that the package imports: `.venv/bin/python3 -c "import machine_learning_engineering; print('ok')"`

## 11.3 Phase 1

1. Rewrite `check_leakage_util.py` to remove ADK imports and use `run_agent + AgentState`.
2. In `debug_util.py`, import `check_leakage_util` and call `check_and_fix_leakage` after a successful `run_and_debug`.
3. In `code_util.py`, make `extract_performance_from_text` robust with regex.
4. In `initialization/agent.py` add the multi-candidate, merger, data-use, and fallback logic.
5. In `initialization/prompt.py` strengthen the prompts.

## 11.4 Phase 2

Create `sub_agents/refinement/__init__.py`, `agent.py`, and `prompt.py`. Implement ablation, summary, plan, implement, and the inner/outer loops. In `debug_util.py`, change `plan_implement` handling to accept the complete improved solution.

## 11.5 Phase 3

Create `sub_agents/ensemble/__init__.py`, `agent.py`, and `prompt.py`. Implement ensemble plan, implement, refine, and loop selection. Make the prompts forbid hallucinated columns.

## 11.6 Phase 4

In `submission/agent.py` implement `_select_final_solution` to choose among refined, ensemble, and initial solutions. Make `_create_submission_workspace` preserve existing files.

## 11.7 Phase 5

In `agent.py` import the new modules and call all four pipelines. Run the pipeline and iteratively fix errors:

```
.venv/bin/python3 -m machine_learning_engineering.agent
```

# 12. Anatomy of a prompt

A prompt is not magic. It is just instructions. Good prompts have three parts:

1. Context: what the LLM needs to know (task description, current code).
2. Task: what the LLM should do (propose models, fix errors, write ensemble code).
3. Format: exactly how the answer should look (JSON list, single code block, exact sentence).

For example, `MODEL_EVAL_INSTR` says: 'You are a Kaggle grandmaster. Here is the task and the model. Write a single-file Python program that trains the model, evaluates on a validation split, and prints Final Validation Performance. Return only a single code block.' The more specific the prompt, the more likely the small model is to follow it. Even then, you must add parsers and fallbacks.

# 13. Example flow through AgentState

To make `AgentState` concrete, here is a simplified trace of what happens during initialization for `task_id 1`:

1. `state['task_description']` = text from `task_description.txt`.
2. `state['task_summary']` = short summary produced by `task_summarization_agent`.
3. `state['init_1_model_1']` = `{'model_name': 'RandomForestRegressor', ...}`.
4. `state['init_1_model_2']` = `{'model_name': 'GradientBoostingRegressor', ...}`.
5. `state['init_code_1_1']` = Python script generated by `model_eval_1_1`.
6. `state['init_code_exec_result_1_1']` = `{'returncode': 0, 'score': 62724.5, ...}`.
7. `state['init_code_1_2']` and its result are stored similarly.
8. `state['base_solution_1']` = the best candidate code.
9. `state['merger_code_1_2']` = code that merges base with candidate 2.
10. `state['train_code_0_1']` = final selected solution after merge/selection/checks.

Every subsequent phase reads and writes keys following the same pattern.

## 14. Glossary

- Agent: a function that sends a prompt to an LLM and processes the response.
- AgentState: a shared Python dictionary that stores all data used by agents.
- Ablation study: an experiment that removes one part of the code to measure its impact.
- Data leakage: using test or validation information during training.
- Ensemble: combining predictions from multiple models to get a better result.
- LLM: large language model, the AI that writes text and code.
- OpenAI-compatible API: an endpoint that uses the same request format as OpenAI.
- Prompt: the text instruction sent to the LLM.
- Runner: the custom module that executes agents and tracks state.
- State key: a string key in AgentState, such as `init_code_1_1`.

## 15. Frequently asked questions

### 15.1 Why does the small model make so many mistakes?

meta-llama-3.1-8b-instruct is much smaller than state-of-the-art models. It is cheaper to run but less reliable at following complex formatting instructions, writing correct code, and avoiding hallucinations. The reconstruction is specifically designed to work around these limitations with robust parsers and fallbacks.

### 15.2 Can I disable refinement or ensemble?

Yes. Set `inner_loop_round=0`, `outer_loop_round=0`, and `ensemble_loop_round=0` in `config.py`. The pipeline will still run initialization and submission.

### 15.3 How do I add a new task?

Create a folder under `machine_learning_engineering/tasks/` with `task_description.txt`, `train.csv`, and `test.csv`. Update `config.py` `task_name` to match the folder name. You may also need to adjust `task_type` and lower depending on the metric.

### 15.4 Where is submission.csv?

After a successful run, `submission.csv` is located at `machine_learning_engineering/workspace/<task_name>/ensemble/final/submission.csv`.

### 15.5 Why not use the original ADK code?

The original code imports `google.adk` and `google.genai`, which are not available in this environment and cannot call the university API. The `runner.py` and rewritten agents replace those dependencies.

## 16. What was NOT implemented

The following items are mentioned in the reconstruction plan but are intentionally left for later phases:

- Web search for model retrieval.
- True parallel execution across `num_solutions > 1`.
- Full rollback agent (`max_rollback_round` is configured but the rollback logic is minimal).
- `skrub` library integration.
- MCTS (Monte Carlo Tree Search) for pipeline configuration search.

These are future improvements once the single-solution path is stable.

## Appendix: key code snippets

This appendix collects the most relevant small changes so you can see exactly what was done without reading every file.

### A. Robust score parsing

```
def extract_performance_from_text(text: str) -> float | None:
    for line in text.splitlines():
        if "Final Validation Performance" in line:
            matches = re.findall(
                r"[-+]?[d*\.]?[d+](?:[eE]([-+]?[d+])?)", line)
            if matches:
                return float(matches[-1])
    return None
```

## B. OpenAI leakage checker outline

```
def check_and_fix_leakage(state, agent_name, max_iterations=2):
    if not state.get("use_data_leakage_checker", False):
        return True
    for _ in range(max_iterations):
        leakage_status, block = _check_leakage_status(state, agent_name)
        if leakage_status != "Yes Data Leakage" or not block:
            return True
        refined_block = _refine_leakage_block(state, agent_name, block)
        new_code = code.replace(block, refined_block, 1)
        _set_code_for_agent(state, agent_name, new_code)
        code_util.evaluate_code(state, agent_name)
```

## C. Ensemble prompt excerpt

```
ENSEMBLE_PLAN_IMPLEMENT_INSTR = """
...
# Required
- Return only the complete ensemble code as a single markdown code block.
- The code should be self-contained and executable as-is.
- Use ONLY the columns that exist in train.csv.
  Do NOT add or assume columns such as 'ocean_proximity'.
- Do not use exit() in the code.
"""
```

## D. Wiring the full pipeline

```
# In machine_learning_engineering/agent.py
initialization_agent_module.run_initialization_pipeline(state)
refinement_agent_module.run_refinement_pipeline(state)
ensemble_agent_module.run_ensemble_pipeline(state)
submission_agent_module.run_submission_pipeline(state)
```

# 17. Concrete example: California Housing end-to-end

This section walks through exactly what happens when you run the rebuilt pipeline on the California Housing task. It shows which scripts are called, which functions are activated, what is stored in AgentState, and which files are created on disk.

## 17.1 Command you run

```
.venv/bin/python3 -m machine_learning_engineering.agent
```

## 17.2 agent.py: entry point

The first function activated is run\_pipeline() in machine\_learning\_engineering/agent.py. It performs four setup steps and then calls the four phase pipelines in order:

1. Create an empty AgentState.
2. Copy task\_name, data\_dir, and workspace\_dir from config.py into state.
3. Set state['total\_tokens\_spent'] = 0.
4. Call the pipelines:
  - initialization\_agent\_module.run\_initialization\_pipeline(state)
  - refinement\_agent\_module.run\_refinement\_pipeline(state)
  - ensemble\_agent\_module.run\_ensemble\_pipeline(state)
  - submission\_agent\_module.run\_submission\_pipeline(state)
5. Call save\_state(state) to write final\_state.json.

## 17.3 Phase 1 - Initialization

`run_initialization_pipeline(state)` in `sub_agents/initialization/agent.py` is the first big pipeline. Here is the exact sequence of function calls:

1. `prepare_task(state)`
  - Copies `DefaultConfig` values into state.
  - Reads `machine_learning_engineering/tasks/california-housing-prices/task_description.txt`.
  - Stores it in `state['task_description']`.
2. `create_workspace(state, task_id=1)`
  - Creates `workspace/california-housing-prices/1`.
  - Creates subdirectories `input` and `model_candidates`.
  - Copies `train.csv` and `test.csv` into `input/`.
3. `run_agent(state, 'task_summarization_agent', ...)`
  - LLM reads `task_description`.
  - Returns a short summary stored in `state['task_summary']`.
4. `run_agent(state, 'model_retriever_agent_1', ...)`
  - LLM proposes two models (e.g. `RandomForestRegressor`, `GradientBoostingRegressor`).
  - `get_model_candidates(...)` parses the response and stores: `state['init_1_model_1']` and `state['init_1_model_2']`.
  - Writes `model_candidates/model_1.txt` and `model_2.txt`.
5. `run_model_eval_for_all_candidates(state, 1)`
  - Calls `debug_util.run_and_debug` for `model_eval_1_1` and `model_eval_1_2`.
  - Each call asks the LLM to write a training script using the proposed model.
  - Generated scripts are saved as `init_code_1.py` and `init_code_2.py`.
  - Results stored in `init_code_exec_result_1_1` and `init_code_exec_result_1_2`.
6. `rank_candidate_solutions(state, 1)`
  - Compares scores, e.g. 62724 vs 60420.
  - Stores `state['base_solution_1']`, `state['best_score_1']`, `state['best_model_id_1']`.
7. `run_merger_and_debug_loop(state, 1)`
  - Merges base solution with the other candidate using agent `merger_1_2`.
  - Generated script saved as `train0_2.py`.
  - Result stored in `merger_code_exec_result_1_2`.
8. `select_best_solution(state, 1)`
  - Chooses the best merged or base solution.
  - Writes `workspace/california-housing-prices/1/train0.py`.
  - Stores `state['train_code_0_1']` and `state['train_code_exec_result_0_1']`.
  - Also stores `best_initial_code` and `best_initial_score` as fallback.
9. `run_check_data_use_and_debug_loop(state, 1)`
  - Verifies `train0.py` uses all provided information.
  - If the LLM thinks something is missing, it returns improved code and the script is re-executed.

## 17.4 Phase 2 - Refinement

`run_refinement_pipeline(state)` in `sub_agents/refinement/agent.py` executes one outer step because `outer_loop_round=1`:

1. Seed the current solution with `state['train_code_0_1']`.
2. `run_ablation_and_debug_loop(state, 1)`
  - Agent name: `ablation_1`.
  - Generated script: `workspace/california-housing-prices/1/ablation_0.py`.
  - Result stored in `ablation_code_exec_result_0_1`.
3. `run_ablation_summary_agent(state, 1)`
  - LLM reads the ablation output and recommends a block to improve.
  - Summary stored in `state['ablation_summary_0_1']`.
4. `run_init_plan_agent(state, 1)`
  - LLM proposes a plan and a `code_block`.
  - Stored in `state['refine_plan_0_0_1']` and `state['refine_code_block_0_0_1']`.
5. `run_init_plan_implement_agent(state, 1)`
  - Agent name: `plan_implement_1`.
  - Generated improved script: `train0_improve0.py`.



- Result stored in `train_code_improve_exec_result_0_0_1`.
- 6. `run_plan_refine_agent(state, 1, inner_iter=1)`
  - Proposes an alternative plan.
- 7. `run_plan_implement_agent(state, 1, inner_iter=1)`
  - Generated alternative script: `train0_improve1.py`.
- Result stored in `train_code_improve_exec_result_1_0_1`.
- 8. `_select_best_improvement(...)` compares all scores and stores `best_refined_code/best_refined_score`.

## 17.5 Phase 3 - Ensemble

`run_ensemble_pipeline(state)` in `sub_agents/ensemble/agent.py` executes one initial plan and one refined plan because `ensemble_loop_round=1`:

1. `_create_ensemble_workspace(state)`
  - Creates `workspace/california-housing-prices/ensemble`.
  - Copies input data.
2. `run_init_ensemble_plan_agent(state)`
  - LLM proposes how to combine solutions.
  - Plan stored in `state['ensemble_plan_0']`.
3. `run_init_ensemble_plan_implement_agent(state)`
  - Generated script: `ensemble/ensemble0.py`.
  - Result stored in `ensemble_code_exec_result_0`.
4. `run_ensemble_plan_refine_agent(state, 1)`
  - Proposes a different ensemble strategy.
5. `run_ensemble_plan_implement_agent(state, 1)`
  - Generated script: `ensemble/ensemble1.py`.
  - Result stored in `ensemble_code_exec_result_1`.
6. `_select_best_ensemble(state)` keeps the best ensemble in `best_ensemble_code/best_ensemble_score`.

## 17.6 Phase 4 - Submission

`run_submission_pipeline(state)` in `sub_agents/submission/agent.py` produces the final deliverable:

1. `_create_submission_workspace(state)`
  - Ensures `ensemble/input` and `ensemble/final` exist.
  - Preserves existing files instead of deleting them.
2. `get_submission_and_debug_agent_instruction` selects the best among:
  - `best_refined_code`
  - `best_ensemble_code`
  - `best_initial_code`
3. `debug_util.run_and_debug(state, 'submission', ...)`
  - LLM adapts the selected solution to train on all data and write `./final/submission.csv`.
  - Generated script: `ensemble/final_solution.py`.
  - Result stored in `submission_code_exec_result`.
4. If execution succeeds, the file `ensemble/final/submission.csv` now contains one prediction per test row.

## 17.7 Files created on disk

After the run completes, these are some of the generated files:

```
workspace/california-housing-prices/1/init_code_1.py
workspace/california-housing-prices/1/init_code_2.py
workspace/california-housing-prices/1/train0_0.py
workspace/california-housing-prices/1/train0_2.py
workspace/california-housing-prices/1/train0.py
workspace/california-housing-prices/1/ablation_0.py
workspace/california-housing-prices/1/train0_improve0.py
workspace/california-housing-prices/1/train0_improve1.py
workspace/california-housing-prices/ensemble/ensemble0.py
```



```
workspace/california-housing-prices/ensemble/ensemble1.py
workspace/california-housing-prices/ensemble/final_solution.py
workspace/california-housing-prices/ensemble/final/submission.csv
workspace/california-housing-prices/final_state.json
```

## 17.8 Final output

The terminal prints something like:

```
=== Pipeline finished ===
```

```
Best score: 245.5250660916751
```

```
Submission return code: 0
```

```
Total tokens spent: 35029
```

A return code of 0 means `final_solution.py` ran successfully and `submission.csv` was created. The Best score is the best validation score found among initial, refined, and ensemble solutions.

## 18. Conclusion

The MLE-STAR reconstruction ports the full multi-agent architecture from Google ADK/Gemini to a custom OpenAI-compatible runner. It covers all six phases of the reconstruction plan, adds robust parsing and fallback mechanisms for the small university model, and validates end-to-end on California Housing. The final pipeline always produces a `submission.csv`, even when individual agents fail or underperform.

The most important lesson is that this kind of system is built incrementally. You add one agent, test it, observe how the small model misbehaves, and add prompts or fallbacks that keep the pipeline moving forward. The code is designed to degrade gracefully: a broken refinement or ensemble never prevents submission because the system can always fall back to the best initial candidate.